



JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 3

Identifying and Defining Objects and Classes

Learning outcomes

Upon completing this topic, you should be able to:

1. Differentiate between Objects and Classes.
2. Identify Objects and Classes from a problem Definition.
3. Translate Classes into database tables.
4. Outline features of generalization, and construct generalization and aggregation models.
5. Examine the different types of object and class relationships.



3.1. Objects and Classes

Coad and Yourdon defined an object as an abstraction of something in a problem domain, reflecting the capabilities of the system to keep information about it, interact with it, or both, for example a Person, A shirt etc. The UML specification gives a definition that illustrates the strength of the connection between the three concepts of an object, instance and class. An object represents a particular instance of a class. Objects that are sufficiently similar to each other are said to belong to the same class. Instance is another word for a single object. The idea that instances belong to a class logically implies that there must be a test that determines to which class an instance belongs.

There are two distinct types of logical similarity which must be tested:



BIT 2114 Object Oriented Analysis and Design

1. All objects in a class share a common set of descriptive characteristics.
2. All objects in a class share a common set of valid behaviors.

For example, if we are interested in developing an Academic Records Information System (ARIS), the table below defines a Student Class template.and each Student object fills in its own unique attribute values.



Attribute Name	Data Type	Value
Name	String	Reagan Okeyo
StudentId	String	123457
Birthdate	Date	January 1st, 1987
Address	String	123 Tom Mboya Street
Major	String	Information Technology
Gpa	Float	3.5
Advisor	???	To be determined
CourseLoad	???	To be determined

Figure 3.1: Student Class Template

Because most systems need to store information in a database, we need to know how to convert a class into a database. The above tabular definition of a class may need to be stored in a database. Classes to be stored in a database can be mapped



onto tables in the following ways:

1. One class is considered as one table.
2. The class' attributes are considered as the columns. If the class has more than one attribute, then additional columns are involved.
3. The instances of the classes are the rows.
4. In case of an object that has cardinality greater than one, and then a new table is formed.

3.2. Analysis - The Rudiments of an Approach

The first reaction to any project with any complexity is "how do we get started?". Where do you begin? The starting point for object-oriented analysis is to identify candidate objects and their



BIT 2114 Object Oriented Analysis and Design

relationships. In fact, this is the be-all and end-all of object-oriented analysis. All we are going to do in the rest of the course is give you some tools to look for objects. But for the present, we will rely on native wit. Begin by looking at the real world. Open your eyes and your ears. The first stage can be a simple brain-storming of the possible objects. One method is to go through all the nouns in any documentation about the world you are analyzing, and considering these as candidate objects. Let us suppose that you are analysing a system for a motor museum. The system is to produce a multimedia guide to the museum. A preliminary list of objects might be: car, bus, vehicle, number plate, exhibit, manufacturer, date of manufacture, value, position, weight, size, photograph, tools, shop, garage, ticket, owner, history. No doubt we could go on. However, the



above list is probably enough to illustrate our needs.

3.2.1. Removing Synonyms and/or Generalizing

The first stage is to try and group together items which are either just different names for the same thing, or whether there is a common generalization. The obvious thing above is to examine the following group car, bus, vehicle, number plate, exhibit, photograph, tools, garage. It would seem that these could all be grouped together under the object exhibit. The question is, can these all be considered to be one object. In this case, the answer is probably yes. In the museum the behavior of cars, photographs, and garages are not radically different, so we can group them together. We can differentiate between the individual objects on the basis of an attribute, say type. We are then



BIT 2114 Object Oriented Analysis and Design

left with the list of candidate objects: exhibit, manufacturer, date of manufacture, value, position, weight, size, shop, ticket, owner, history.

3.2.2. Look for Attributes

Attributes are themselves objects, but with trivial behavior - essentially all you do is record their value. Considering the above list, the following are probably attributes of exhibit: exhibit, date of manufacture, value, position, weight, size, owner So we have the exhibit object as: And we are left with the candidate objects: exhibit, manufacturer, shop, ticket, history We might have considered manufacturer as an attribute, but it may be useful to separate out manufacturer information later in the design. So we keep it for the time-being. The time to fold it in as



BIT 2114 Object Oriented Analysis and Design

an attribute is when we find it has no discernible behavior of its own.

3.2.3. Irrelevant Objects

Sometimes an object is irrelevant because of what you are designing the system for. Here the ticket object may be very relevant for a sales desk system, but not for a multimedia system.

Likewise, the shop is irrelevant. So we strike these out and we are left with the following objects. exhibit, manufacturer, history We have these objects potentially linked in the following way.

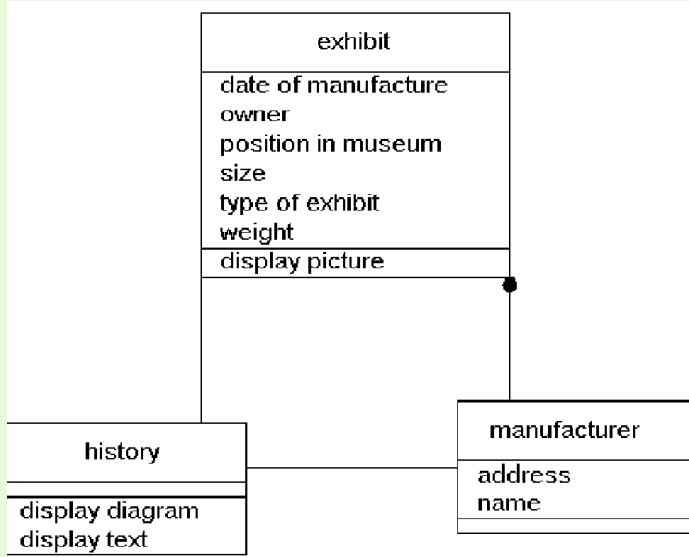


Figure 3.2: Exhibit Initial Class Diagram

Both exhibits and manufacturers may have histories. By fac-



BIT 2114 Object Oriented Analysis and Design

toring out manufacturers, the system may be able to trace from an exhibit not only its history, but also that of its manufacturer, and maybe allow someone to search for all the objects in the museum held by a given manufacturer.

3.3. The Process of Development

The approach to development recommended here is an iterative one. It involves repeated refinement of the object model. The process needs to be controlled by an appropriate project management process, involving reviews and check-pointing.



BIT 2114 Object Oriented Analysis and Design

The approach to development recommended here is an iterative one. It involves repeated refinement of the object model. The process needs to be controlled by an appropriate project management process, involving reviews and checkpointing.

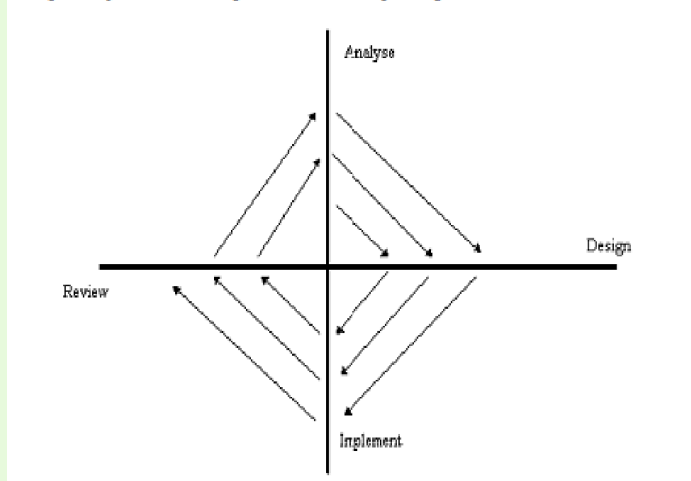


Figure 3.3: Process of Development

There are a variety of techniques for driving this cycle:



Prototyping

The design of the process progresses by the construction of a series of prototypes. The prototypes are constructed using the most productive tools in terms of speed of development. The final system, when constructed, is functionally equivalent to the final prototype, but it is implemented in the most suitable technology for use of the system. For example, a patient monitoring system, requiring lots of graphical displays from a digital feed, might be prototyped in a spreadsheet because the spreadsheet has very powerful graphics. However, when the system is built it would have to be implemented to run on a specific computer with a limited display and limited amount of memory, without a spreadsheet license.

Using iterative design methods, a series of prototypes could



BIT 2114 Object Oriented Analysis and Design

be developed using a spreadsheet until functionality is finally agreed. The final prototype would then be used as an active specification for the final system which might be written in C.

Prototyping is useful in the following contexts:

- at the beginning of a project
- in research-oriented projects where there needs to be some exploration of different ways of solving problems
- where there is a need to clearly define interfaces
- where the client needs reassurance that progress is being made

The weaknesses of prototyping are:

- the prototype is often taken on as a final system
- there can be difficulty explaining a long lag between first



prototype and final system

- inevitably there will be some differences between the final system and the final prototype because of different interface mechanisms.

Evolutionary Development

This process involves a staged implementation of the system. Analysis and design of the next stage is after successful implementation of the previous stage. Evolutionary development is useful if:

- a partial implementation of the proposed system is of some use
- it is not replacing another system which cannot be integrated appropriately at each stage of development



BIT 2114 Object Oriented Analysis and Design

The weaknesses of evolutionary development are:

- disruption caused by each stage of implementation
- difficulty controlling the end-point of development

Boxing

Computer systems development is constrained by money, time and personnel available to develop, implement and install. The idea of boxing is to drive the iterative process of development by check pointing at key stages according to one of the three constraints. Time-boxing involves setting a time limit on each iteration of the cycle. As the time limit nears, the system is implemented, irrespective of whether the planned functionality is fully implemented. The system is reviewed, and the plans for the next stage take into account what has been achieved and what



BIT 2114 Object Oriented Analysis and Design

has been learnt. Money-boxing is similar to time-boxing, except that the reviews are triggered by expenditure limits. Personnel-boxing involves controlling the process using the time of certain personnel. Boxing has an advantage of driving the system, and forcing changes of plan as the system development progresses.

Charters

Rather than produce a contract defining all functionality to be delivered in a cycle, it is possible instead to construct a charter. A charter prioritizes the functionality which should be delivered in the cycle. The charter can over-specify what is expected in the cycle, in case progress is faster than anticipated.



Why Object Modeling Lends Itself to Iterative Methods

The problem with traditional systems developments has been that notations differ in each of the phases of development. There is then a translation problem and consistency problem at each phase transition. Consequently changes upstream have always been resisted. Object modeling uses the same notation for both analysis and design, and if object-oriented languages and databases are used, the notation persists through to implementation. Design is essentially a fleshing out of the analysis, by further refining and adding detail to the objects produced by the analysis, and by adding new objects, say for the purposes of providing interfaces. Object modeling, by using a consistent notation in each phase, results in more productive CASE tools which can generate the code outlines for the implementation.



Generalization

In the UML specification (OMG, 2001) generalization is formally defined as: a taxonomic relationship between a more general element and a more specific element. The more specific element is fully consistent with the more general element and may contain additional information. The main application for generalization in object-orientation is to describe relationship of similarity between classes. A second benefit comes from the relative ease with which a hierarchy can be extended to fit a changing picture.

Other features of generalization not mentioned in the definition are:

- Inheritance: The mechanism of implementing generalization and specialization in an object-oriented programming language.



BIT 2114 Object Oriented Analysis and Design

- Transitive operation: The relationship between two elements at adjacent levels of hierarchy ‘carries over’ to all more specialized levels.
- Disjoint nature of generalization hierarchies: an object cannot belong to two classes, for example, a cat cannot be both a mammal and a reptile.

Message Passing

In an object oriented systems, objects communicate with each other by sending messages. Message passing allows objects to encapsulate their internal details from other parts of the system, thus minimizing the knock-on effect of any changes to the design or implementation.



Aggregation

We can make up objects out of other objects. This is known as aggregation. The behavior of the bigger object is defined by the behavior of its component parts, separately and in. Here is a simple example of a juggler:

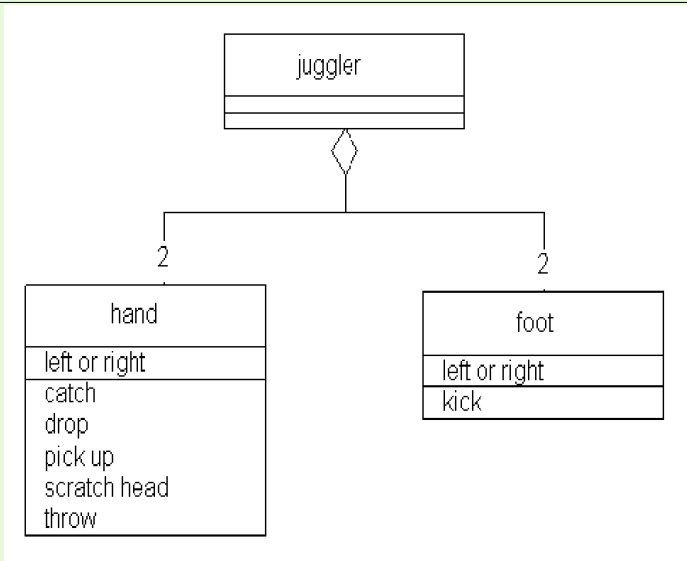


Figure 3.4: Aggregation Example

A juggler has two hands and two feet. He or she uses hands



BIT 2114 Object Oriented Analysis and Design

to catch, drop, pick up and throw a ball, and perhaps from time to time scratch his or her head. He or she may also kick a ball with his or her foot. By analyzing our juggler object and breaking it down into component objects, we now have a better understanding of our object. Also, we have discovered two new operations, scratch head and kick. The diamond in the diagram indicates that one object is made up of another object. The numbers are indicative of how many - more will be said about this shortly. Now the behavior of a juggler is entirely defined by the behavior of his or her hands and feet. Of course, real jugglers are made of quite a bit more, but for the purposes of considering their juggling skills we can focus on just the bits of them that are involved in juggling. Hands and feet could be broken down into their constituent parts, say palms and fingers, soles and toes.



BIT 2114 Object Oriented Analysis and Design

However, that does not seem to help us to understand juggling, so the decomposition above is probably enough.

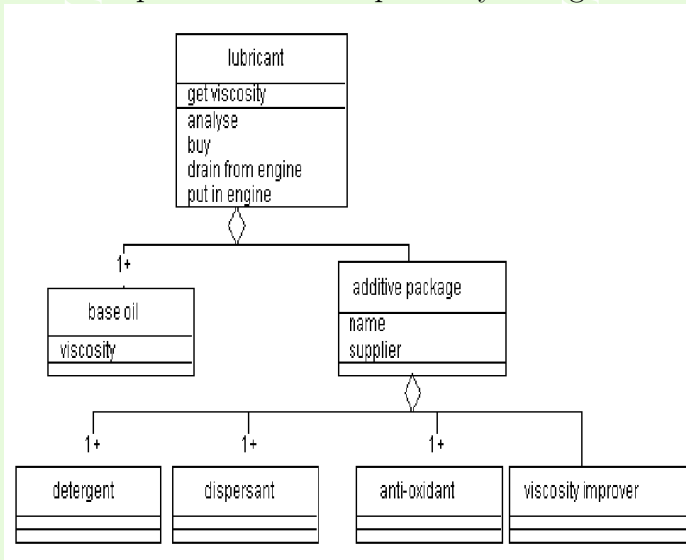




Figure 3.5: Another Aggregation Example

A car engine lubricant is made up of a number of base oils blended with an additive package. The additive package consists of one or more detergents to keep engine surfaces clean, one or more dispersants to suspend particles in the oil to be carried to the filter, one or more anti-oxidants to slow up the thermal decay of the oil, and a viscosity improver to control the viscosity of the oil at different temperatures. A research scientist will experiment with different oils by running engines containing the oils and analyzing the effect of the engines on the oils. One of the important analysis and design tools we have is the breakdown of complex objects into their constituent parts. It provides a meaningful and sensible decomposition, and it provides scope for re-use of components. Further, the constituent components



BIT 2114 Object Oriented Analysis and Design

are often easier to design than large, complex components - this is the thesis on which the early ideas of structured programming were based.

Delegation

The behavior of an object which is made up of other objects is usually defined by the behavior of the smaller objects. For example:

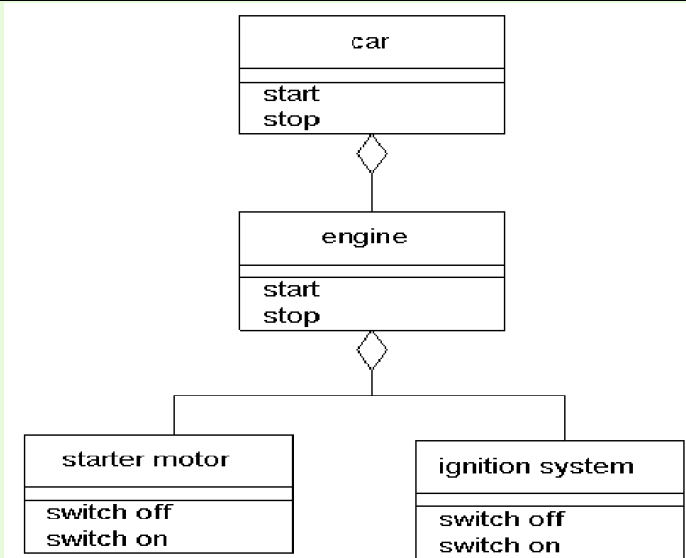


Figure 3.6: Delegation Example

To start a car, you start the engine. Thus the start operation



BIT 2114 Object Oriented Analysis and Design

on the car involves calling the start operation on the engine. This is known as delegation. The engine then will switch on the ignition system, switch on the starter motor then switch off the starter motor. This is further delegation. To stop the car, there will be a call to stop the engine, which in turn will make a further call to switch off the ignition. You may read elsewhere about the benefits of multiple inheritance. Most of the features of multiple inheritance can be simulated using delegation, with safer consequences. However, the arguments for and against multiple inheritance (inheriting from more than one parent) are lengthy and can be side-stepped for now.



Relationships

Objects can be related in other ways than by inheritance and aggregation. Any relationship between real world objects can be modeled: cats eat canaries, dogs bite postmen, the woodcutter murders the wolf, cars run over little old ladies, employees work for organizations, patients visit hospitals, patients stay in hospitals.

One to one relationships

In a one-to-one relationship, one object is associated with exactly one of its related objects. This is modeled by a straight line drawn between the objects. If the relationship is one-way, then an arrow is used to indicate the direction. The line can be labeled.

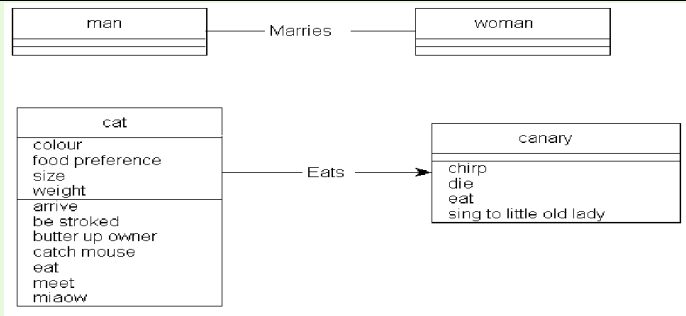


Figure 3.7: One to One Relationship

Thus a man marries one woman (at a time) and a woman marries one man (at a time). A cat eats one canary (before being battered to death by the little old lady who owned the canary). Canaries do not (in general) eat cats, so the eats relationship is one way.



One to Many Relationships

Sometimes one object can be related to many objects. This is indicated by different marks at the end of the line.

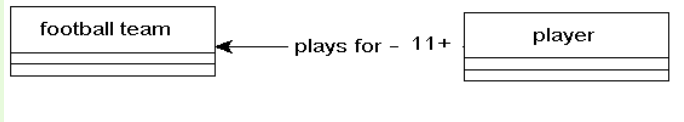


Figure 3.8: One to Many Relationship

A player plays for one football team. There are at least 11 players for a given football team. Football teams do not play for players.

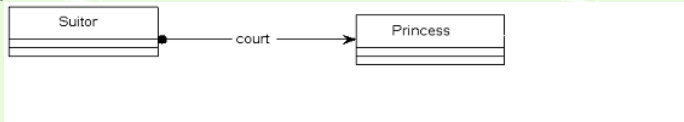


Figure 3.9: One to Many Relationship



Zero or more suitors court the Princess. The black dot at the end of the line indicates zero or more.



Figure 3.10: One to Many Relationship

Before an adequate suitor comes along, as is well known, a princess will kiss at least one frog, and possibly many more if she gets a taste for them. Frogs, being well mannered creatures and not wishing to appear in the gossip columns, never let more than one Princess kiss them.

Many to Many Relationships

Sometimes objects at either end of a relationship may be related to many objects at the other end.

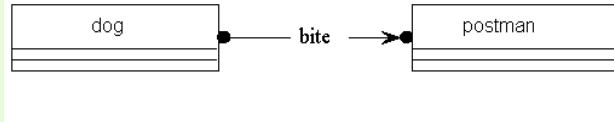


Figure 3.11: Many to Many Relationship

A dog may bite zero or more postmen. A postman may be bitten by zero or more dogs.

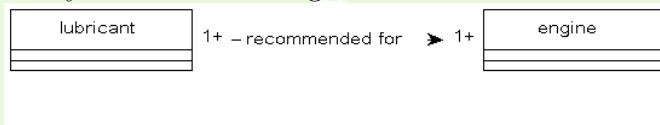


Figure 3.12: Many to Many Relationship

A lubricant is recommended for at least one engine. An engine has at least one lubricant recommended for it.



3.4. Object Models

We now have the notation to describe quite complicated systems. The process of object-oriented analysis and design is one of elaborating an object model, increasing its detail and scope until enough is known to construct a computer system (if indeed that is what is wanted).

An example is one to describe patient referrals by GP's to specialists.

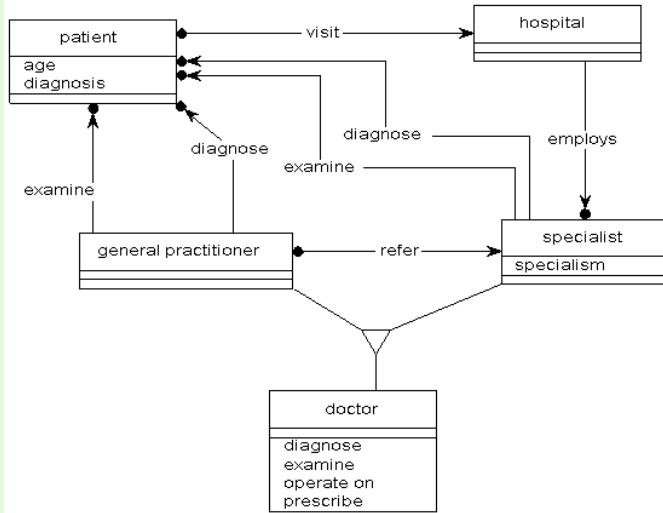


Figure 3.13: Object Model

Another example is for a lift system:

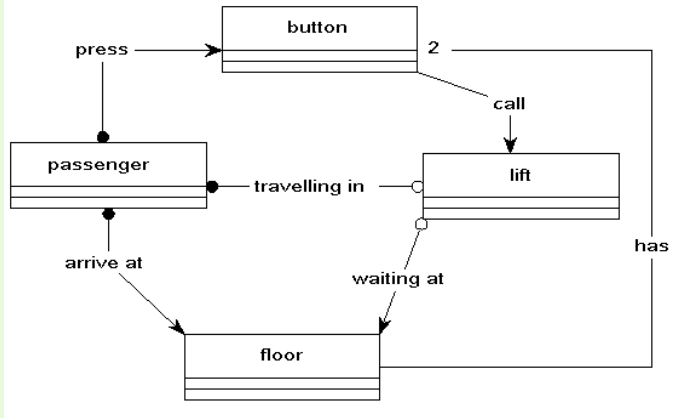


Figure 3.14: Object Model

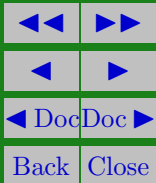
The object model is the principal output of an analysis and design process. The distinction between analysis and design is much greyer in object-oriented development. Essentially it is one of detail. Analysis usually omits concerns about how a system



JKUAT SODeL
©2014

BIT 2114 Object Oriented Analysis and Design

is to be developed, and some of the objects may not be fully decomposed. The object model is the central pillar of a analysis or design. It defines the computable elements.





Learning Activities

Construct an object model for one of the following: the football league, star wars, a personnel system, Cinderella, an automatic washing machine, a video hire shop, a kitchen.

Revision Questions

EXERCISE 1.  Briefly explain why it is important to categorise objects into classes

Example  Differentiate between include and extend relationship for use cases

Solution: for revision



EXERCISE 2.  Briefly explain the concept of an object and hence describe three characteristics of an object.



References and Additional Reading Materials

1. TCAD Journal: The Journal of Technology, Computer Aided Design.
 - <http://www.ieee.org/journal/tcad/>
2. CAD/CAM Publishing: books, magazines, industry links
<http://www.cadcamnet.com/>